README — ParkirKu

Pencatat Parkir Kampus Surabaya · Tugas Individu #1 Algoritma dan Pemrograman (BBK1AAB4) 2026

Nama lengkap	[NAMA LENGKAP]
NIM	[NIM]
Kelas	[KELAS]
Dosen pengampu	[NAMA DOSEN]
Berkas program	`parkir.py` (Python 3.12, satu berkas, 284 baris)

Daftar Isi

1. Identitas Diri (di atas)
2. Deskripsi Program
3. Dekomposisi Masalah
4. Flowchart & Pseudocode Tahap 3
5. Tabel Penelusuran
6. Tabel Kasus Uji
7. Catatan Galat
8. Refleksi
9. Pengungkapan Penggunaan AI

2. Deskripsi Program

ParkirKu adalah program teks (console) yang membantu petugas gerbang mencatat setiap kendaraan yang keluar dari area parkir kampus dalam satu hari. Untuk setiap kendaraan, petugas memasukkan plat, jenis (motor/mobil), jam dan menit masuk-keluar, status kartu civitas, serta status tiket. Program menolak masukan yang salah dan meminta ulang sampai benar, lalu menghitung durasi, jam ditagih, tarif dasar (dengan batas gratis 15 menit dan batas maksimum per hari), diskon 20% kartu civitas, denda tiket hilang, dan total bayar, kemudian mencetak struk. Pencatatan berulang sampai petugas mengetik `SELESAI`, setelah itu program mencetak rekap harian lengkap dengan grafik batang teks.

Program disusun dalam lima tahap yang berurutan dan hanya memakai materi pekan 1–6 (variabel, tipe data, `input()`, operator, `if/elif/else`, `and/or/not`, `while`, `for` dan `range()`), tanpa `def`, list, tuple, dictionary, `import`, `try/except`, `break`, dan `continue`. Semua angka aturan disimpan sebagai konstanta huruf kapital di bagian atas program sehingga rumus tidak berisi "angka ajaib". Selain fitur wajib, ada beberapa fitur tambahan yang relevan untuk petugas:

- Plat kosong ditolak; plat dan kata `SELESAI` dikenali tanpa membedakan huruf besar/kecil (`selesai` juga menghentikan program).
- Isian jam/menit yang berisi huruf (mis. `8a`) tidak membuat program galat: diperiksa dengan `isdigit()` sebelum `int()`, tanpa `try/except`.
- Jawaban jenis dan ya/tidak menerima huruf besar (`MOTOR`, `Ya`) dan spasi berlebih.
- Rekap tambahan: pendapatan dari motor dan mobil, serta total diskon yang diberikan; grafik batang diberi angka jumlahnya.

3. Dekomposisi Masalah

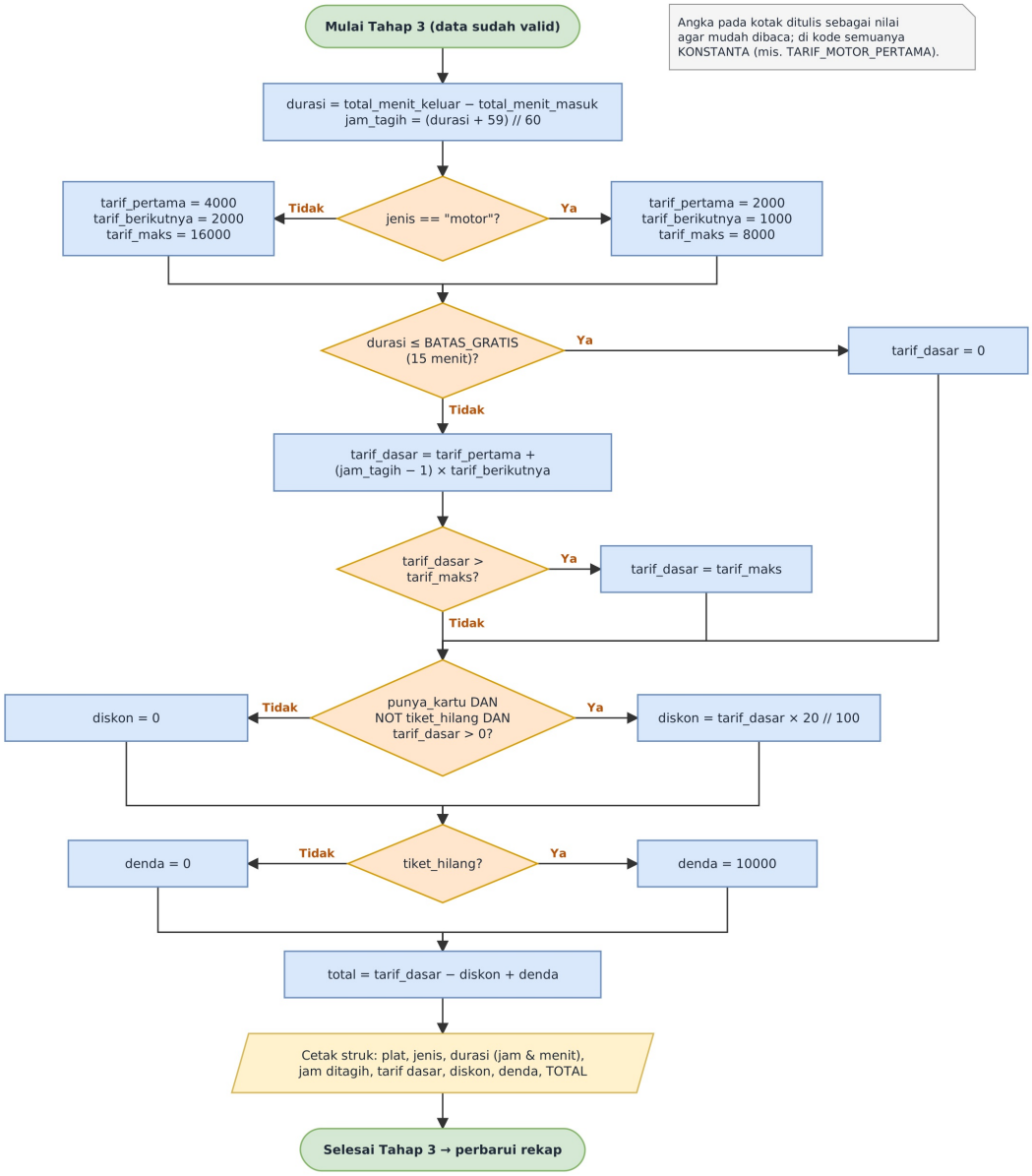
Masalah besar "mencatat parkir satu hari" dipecah menjadi sub-masalah berikut. Baris bertanda **TERSIRAT** adalah sub-masalah yang tidak ditulis eksplisit di soal tetapi harus diselesaikan agar program benar.

No	Sub-masalah	Plan pemrograman	Letak di program
1	Menyimpan semua angka aturan tarif dan kata penanda di satu tempat	Konstanta bernama (huruf kapital)	Tahap 1, baris 21–53
2	Mencetak tabel tarif 1–8 jam yang sudah dibatasi maksimum	`for` + `range()`, penjaga batas atas (*capping*)	Tahap 1, baris 61–69
3	Memberi nilai awal semua variabel rekap sebelum perulangan	Inisialisasi pencacah/akumulator = 0, ekstrem = −1	Baris 77–90
4	Mengulang pencatatan tanpa tahu jumlah kendaraan	Sentinel (*priming read* + baca ulang di akhir blok)	Tahap 4, baris 98, 103, 242

5	Menolak jenis selain motor/mobil	Validasi `while` + kondisi majemuk `and`	Tahap 2, baris 111–114
6	Menolak jam di luar 0–23 dan menit di luar 0–59	Validasi `while` + variabel penanda `sah` + `or`	Tahap 2, baris 117–132
7	Menolak waktu keluar yang lebih awal dari waktu masuk	Penjaga rentang, `if/elif/else`	Tahap 2, baris 135–151
8	Menolak jawaban selain ya/tidak (kartu & tiket)	Validasi `while` + `and`	Tahap 2, baris 154–165
9	Menghitung durasi dan jam ditagih (bulat ke atas)	Konversi ke total menit, `//` dan `%`	Tahap 3, baris 170–171
10	Memilih tarif sesuai jenis	`if/else`	Tahap 3, baris 174–181
11	Tarif dasar: gratis ≤ 15 menit, normal, atau batas maksimum	`if/else` bersarang, penjaga batas	Tahap 3, baris 185–190
12	Diskon 20% hanya jika kartu DAN tiket tidak hilang DAN tarif > 0	Kondisi majemuk `and` + `not`	Tahap 3, baris 193–196
13	Denda tiket hilang (tetap berlaku walau gratis), total, struk	`if/else`, ekspresi aritmetika	Tahap 3, baris 199–215
14	Menghitung jumlah kendaraan, motor, mobil	Pencacah	Baris 219, 221–226
15	Menghitung kendaraan gratis, berdiskon, kena denda	Penyaringan + pencacah	Baris 227–233
16	Total pendapatan (dan per jenis)	Akumulator	Baris 220, 223, 226
17	Parkir terlama & bayar tertinggi (seri → yang lebih dulu)	Nilai ekstrem dengan `>` (bukan `>=`)	Baris 234–239
18	Rata-rata dan rekap tanpa pembagian nol	Penjaga data kosong, akumulator ÷ pencacah (`//`)	Tahap 5, baris 253–271
19	Grafik batang teks motor dan mobil	`for` + `range()`, `print(end="")`	Tahap 5, baris 273–282
20	**TERSIRAT** Membandingkan waktu masuk–keluar dengan benar (mis. 10:30 vs 10:29, jam sama tetapi menit lebih kecil)	Ubah ke total menit lalu bandingkan satu angka	Baris 132, 144, 148
21	**TERSIRAT** Mencegah program galat bila jam/menit berisi huruf	Penjaga `isdigit()` sebelum `int()`	Baris 121, 139
22	**TERSIRAT** Plat kosong tidak boleh dicatat; `selesai` huruf kecil tetap penanda	Validasi `while`, normalisasi `.strip().upper()`	Baris 98–101, 242–245
23	**TERSIRAT** Durasi 0 menit (masuk = keluar) harus sah dan gratis	Kondisi `<` (bukan `<=`) pada penjaga waktu keluar	Baris 148, 185
24	**TERSIRAT** Menentukan arti "parkir gratis" bila tiket hilang	Penyaringan berdasarkan `tarif_dasar == 0` (sesuai aturan 3.0)	Baris 227
25	**TERSIRAT** Nilai awal ekstrem yang benar walau semua durasi/total = 0	Nilai awal −1 (lebih kecil dari nilai sah terkecil)	Baris 87–90
26	**TERSIRAT** Menampilkan durasi dalam jam dan menit	`durasi // 60` dan `durasi % 60`	Baris 210, 269–270

4. Flowchart & Pseudocode Tahap 3 (Hitung Biaya)

Flowchart lengkap semua tahap (6 halaman) tersedia dalam berkas `flowchart.drawio` dan dapat dibuka di *app.diagrams.net*. Angka di kotak ditulis sebagai nilai agar mudah dibaca; di kode semuanya berupa konstanta.



Pseudocode Tahap 3

```
MULAI TAHAP 3                          { semua masukan sudah valid dari Tahap 2 }
  durasi    <- total_menit_keluar - total_menit_masuk
  jam_tagih <- (durasi + 59) div 60     { pembulatan ke atas ke jam penuh }

  JIKA jenis = "motor" MAKA
      tarif_pertama <- TARIF_MOTOR_PERTAMA ; tarif_berikutnya <- TARIF_MOTOR_BERIKUTNYA
      tarif_maks    <- TARIF_MOTOR_MAKS
  SELAIN ITU
      tarif_pertama <- TARIF_MOBIL_PERTAMA ; tarif_berikutnya <- TARIF_MOBIL_BERIKUTNYA
      tarif_maks    <- TARIF_MOBIL_MAKS

  JIKA durasi <= BATAS_GRATIS_MENIT MAKA
      tarif_dasar <- 0
  SELAIN ITU
      tarif_dasar <- tarif_pertama + (jam_tagih - 1) * tarif_berikutnya
      JIKA tarif_dasar > tarif_maks MAKA tarif_dasar <- tarif_maks

  JIKA punya_kartu DAN BUKAN tiket_hilang DAN tarif_dasar > 0 MAKA
      diskon <- tarif_dasar * PERSEN_DISKON div 100
  SELAIN ITU
      diskon <- 0

  JIKA tiket_hilang MAKA denda <- DENDA_TIKET_HILANG SELAIN ITU denda <- 0

  total <- tarif_dasar - diskon + denda
  TAMPILKAN struk: plat, jenis, durasi (durasi div 60 jam, durasi mod 60 menit),
                   jam_tagih, tarif_dasar, diskon, denda, total
SELESAI TAHAP 3
```

Mengapa batas maksimum diterapkan dengan "potong" setelah rumus? Rumus yang sama persis dipakai di tabel Tahap 1, sehingga angka di tabel dijamin sama dengan angka yang ditagih. Urutan ini juga membuat kasus "tepat di batas" (motor 7 jam = Rp 8000) tidak terpotong karena kondisinya `>`, bukan `>=`.

5. Tabel Penelusuran (Data 5.1)

Penelusuran dihitung dengan tangan lebih dulu, baru kemudian dibandingkan dengan keluaran program.

5.1 Perhitungan tangan per kendaraan

Plat	Langkah perhitungan
A 1111 AA motor 13:00–13:40, kartu tidak, tiket tidak	masuk = 13×60+0 = 780; keluar = 13×60+40 = 820; durasi = 40 menit · jam_tagih = (40+59)//60 = 99//60 = **1** · 40 > 15 → tarif = 2000 + (1−1)×1000 = **2000** (≤ 8000) · tidak punya kartu → diskon 0 · denda 0 · total = **2000**
B 2222 BB mobil 06:50–09:10, kartu ya, tiket tidak	masuk = 410; keluar = 550; durasi = **140** menit (2 jam 20 menit) · jam_tagih = 199//60 = **3** · tarif = 4000 + 2×2000 = **8000** (≤ 16000) · kartu ya, tiket tidak hilang, tarif > 0 → diskon = 8000×20//100 = **1600** · denda 0 · total = **6400**
C 3333 CC motor 15:00–15:15, kartu tidak, tiket tidak	masuk = 900; keluar = 915; durasi = **15** menit · jam_tagih = 74//60 = **1** · 15 ≤ 15 → tarif dasar = **0** (tepat batas gratis) · diskon 0 · denda 0 · total = **0**

5.2 Tabel penelusuran variabel

Kendaraan ke-	durasi	jam_tagih	tarif_dasar	diskon	total	jumlah	total_pendapatan	durasi_terlama
Awal	-	-	-	-	-	0	0	-1
1 (A 1111 AA)	40	1	2000	0	2000	1	2000	40
2 (B 2222 BB)	140	3	8000	1600	6400	2	8400	140
3 (C 3333 CC)	15	1	0	0	0	3	8400	140

Variabel rekap lain di akhir: motor 2, mobil 1, gratis 1 (C 3333 CC), diskon 1 (B 2222 BB), denda 0, rata-rata = 8400 // 3 = **2800**, terlama **B 2222 BB (2 jam 20 menit)**, bayar tertinggi **B 2222 BB (Rp 6400)**. Pada kendaraan ke-3, `durasi_terlama` tetap 140 karena 15 > 140 bernilai salah.

5.3 Perbandingan dengan hasil program

Besaran	Hitung tangan	Hasil program	Status
Total A 1111 AA / B 2222 BB / C 3333 CC	2000 / 6400 / 0	2000 / 6400 / 0	**Sama**
Jam ditagih (1 / 2 / 3)	1 / 3 / 1	1 / 3 / 1	**Sama**
Total pendapatan, rata-rata	8400, 2800	8400, 2800	**Sama**
Parkir terlama, bayar tertinggi	B 2222 BB (2j 20m), B 2222 BB (6400)	B 2222 BB (2j 20m), B 2222 BB (6400)	**Sama**

Mengapa nilai awal `durasi_terlama` adalah −1, bukan 0?
Pola nilai ekstrem bekerja dengan membandingkan setiap data baru terhadap "juara sementara" memakai `durasi > durasi_terlama`. Nilai awal harus *lebih kecil dari semua nilai sah* agar kendaraan pertama pasti menggantikannya. Durasi yang sah dimulai dari **0 menit** (masuk dan keluar pada menit yang sama, misalnya 23:59–23:59). Jika nilai awal 0 dan kendaraan pertama berdurasi 0, maka `0 > 0` bernilai salah, sehingga `plat_terlama` tetap kosong dan rekap mencetak plat kosong. Dengan nilai awal −1, perbandingan `0 > -1` benar, sehingga kendaraan pertama selalu tercatat. Alasan yang sama dipakai untuk `bayar_tertinggi = -1`, karena total bayar juga bisa Rp 0. Kasus ini sudah diuji (kasus uji B6). Operator `>` (bukan `>=`) sekaligus memastikan bila seri, kendaraan yang lebih dulu tercatat yang dipertahankan.

6. Tabel Kasus Uji

Semua kasus dijalankan pada program akhir. Kolom "Hasil program" adalah keluaran sebenarnya. Kategori: **N** = normal, **B** = batas, **T** = tidak valid.

ID	Masukan	Hasil yang diharapkan	Hasil program
N1	L 1234 AB, motor, 08:00–11:20, kartu ya, tiket tidak	Durasi 3j 20m, jam 4, tarif 5000, diskon 1000, total Rp 4000	**Total Rp 4000** ✓
N2	N 7788 IJ, mobil, 12:00–14:10, tidak, tidak	Jam 3, tarif 8000, total Rp 8000	**Total Rp 8000** ✓
N3	A 1111 AA, motor, 13:00–13:40, tidak, tidak	Jam 1, total Rp 2000	**Total Rp 2000** ✓
N4	Kelima data uji contoh (3.7), lalu SELESAI	5 kendaraan (3 motor, 2 mobil), 1 gratis, 2 diskon, 1 denda, Rp 30400, rata Rp 6080, terlama S 9012 EF, tertinggi L 3456 GH	**Semua sama persis** ✓
B1	W 5678 CD, mobil, 09:00–09:15 (tepat 15 menit)	Tarif dasar 0, total Rp 0, terhitung gratis	**Total Rp 0, gratis 1** ✓
B2	Motor 00:00–00:16 (16 menit, 1 menit lewat batas gratis), kartu ya	Jam 1, tarif 2000, diskon 400, total Rp 1600	**Total Rp 1600** ✓
B3	Motor 10:00–11:00 (tepat 60 menit) lalu motor 10:00–11:01 (61 menit)	60 menit → jam 1, Rp 2000; 61 menit → jam 2, Rp 3000	**Rp 2000 dan Rp 3000** ✓
B4	Motor 08:00–15:00 (7 jam, tepat Rp 8000 = maksimum)	Tarif 8000, tidak terpotong	**Tarif Rp 8000** ✓
B5	S 9012 EF motor 07:30–19:45, kartu ya (13 jam)	Rumus 14000 dipotong ke 8000, diskon 1600, total Rp 6400	**Total Rp 6400** ✓
B6	Mobil 23:59–23:59 (durasi 0), kartu ya, tiket hilang	Jam 0, tarif 0, tanpa diskon, denda 10000, total Rp 10000; terlama tercatat (0 jam 0 menit)	**Total Rp 10000, terlama X 1** ✓
B7	Mobil 00:00–23:59 (durasi maksimum sehari)	Jam 24, tarif dipotong 16000	**Total Rp 16000** ✓
B8	Dua motor masing-masing 1 jam, Rp 2000 (seri)	Terlama & tertinggi = kendaraan pertama	**P1 untuk keduanya** ✓
B9	Langsung `SELESAI` (data kosong)	"Belum ada kendaraan yang tercatat.", tanpa rata-rata/ekstrem, tidak galat	**Sesuai, tidak galat** ✓
T1	Jenis `sepeda`	"Jenis harus motor atau mobil.", tanya ulang	**Sesuai** ✓
T2	Jam masuk 25, menit 0; lalu jam −1; lalu menit 60	"Waktu tidak valid." / pesan angka, tanya ulang jam & menit	**Sesuai** ✓
T3	Masuk 08:00, keluar 07:00	"Waktu keluar tidak boleh sebelum waktu masuk."	**Sesuai** ✓
T4	Masuk 10:30, keluar 10:29 (jam sama, menit lebih kecil)	Ditolak; setelah 10:45 → durasi 15, gratis	**Ditolak, lalu Rp 0** ✓
T5	Jam masuk `abc`; kartu `mungkin`; jenis `MOTOR`, jawaban `Ya` / `TIDAK`	Huruf ditolak tanpa galat; huruf besar diterima	**Sesuai, tidak galat** ✓
T6	Plat kosong / hanya spasi, lalu `selesai`	"Plat nomor tidak boleh kosong.", lalu berhenti	**Sesuai** ✓

7. Catatan Galat

Ketiga galat di bawah ini muncul saat program dikembangkan dan diuji. Pesan galat disalin dari keluaran Python yang sebenarnya.

Jenis	Gejala	Cara menemukan	Perbaikan
Sintaks	Program tidak mau berjalan sama sekali: `SyntaxError: expected ':'` pada `if tiket_hilang`.	Galat sintaks terdeteksi *sebelum* baris pertama dijalankan. Python menunjuk baris dan posisi (tanda `^`) yang salah.	Menambahkan titik dua: `if tiket_hilang:`.
Runtime	Program berjalan, tetapi berhenti di tengah dengan `ZeroDivisionError: integer division or modulo by zero` saat petugas langsung mengetik `SELESAI`.	Galat muncul hanya pada masukan tertentu (data kosong). Traceback menunjuk baris `total_pendapatan // jumlah_kendaraan`.	Rata-rata dipindah ke dalam cabang `else` dari penjaga `if jumlah_kendaraan == 0`.

Runtime (dicegah)	`int("8a")` menghasilkan `ValueError: invalid literal for int()`.	Mencoba mengetik huruf pada isian jam.	Penjaga `isdigit()` sebelum `int()` (tanpa `try/except`).
Logika	Program berjalan tanpa pesan galat, tetapi angka salah: L 1234 AB menjadi Rp 3200 (seharusnya 4000), L 3456 GH Rp 11000 (seharusnya 12000), total Rp 26600 (seharusnya 30400).	Membandingkan hasil dengan data uji 3.7 dan tabel penelusuran tangan. Struk menampilkan "Jam ditagih : 0" untuk durasi 16 menit, sehingga rumus menjadi 2000 + (0−1)×1000 = 1000.	Jam ditagih harus dibulatkan ke atas: `(durasi + 59) // 60`, bukan `durasi // 60`.

Cara membedakan ketiganya: galat **sintaks** menghentikan program sebelum berjalan (aturan penulisan dilanggar); galat **runtime** terjadi saat program sedang berjalan dan muncul traceback (operasi tidak mungkin dilakukan pada nilai tertentu); galat **logika** tidak memunculkan pesan apa pun, program selesai normal tetapi hasilnya salah, sehingga hanya bisa ditemukan dengan membandingkan terhadap hasil yang diharapkan (tabel penelusuran dan kasus uji).

8. Refleksi

Konsep yang paling saya pahami adalah pola validasi dengan `while` dan perulangan berpenanda: kondisi `while` sendiri menjelaskan kapan perulangan berhenti, jadi saya tidak memerlukan `while True` dan `break`. Saya juga baru menyadari pentingnya nilai awal pada pola nilai ekstrem; satu angka (0 atau −1) bisa membuat rekap salah tanpa pesan galat. Bagian yang masih terasa sulit adalah menyusun kondisi majemuk agar tidak terbalik, misalnya kapan memakai `and` dan kapan `or` pada validasi "bukan motor dan bukan mobil". Saya juga masih perlu berlatih menemukan galat logika lebih cepat, karena galat jenis ini tidak memberi petunjuk apa pun selain angka yang salah.

9. Pengungkapan Penggunaan AI

Saya memakai asisten AI **Claude (Anthropic)** dalam tugas ini. Bagian yang dibantu AI: penyusunan kode `parkir.py`, kerangka README ini (dekomposisi, pseudocode, tabel kasus uji), dan flowchart `flowchart.drawio`. Semua hasil sudah saya jalankan sendiri dengan data uji soal dan data penelusuran, dan saya pelajari baris demi baris agar dapat saya jelaskan saat audit.

Saran yang saya pakai	Saran / alternatif yang saya tolak dan alasannya
• Membandingkan waktu dalam total menit, bukan jam dan menit terpisah. • Variabel penanda `waktu_masuk_sah` untuk validasi beberapa kondisi sekaligus. • Penjaga `isdigit()` agar huruf pada isian angka tidak membuat program galat. • Nilai awal −1 untuk variabel ekstrem dan operator `>` untuk kasus seri. • Memotong tarif ke maksimum dengan rumus yang sama seperti tabel Tahap 1.	• `while True` + `break` untuk perulangan: dilarang soal, dan kondisinya jadi tidak terlihat di baris `while`. • `try/except ValueError` untuk masukan huruf: dilarang soal; diganti `isdigit()`. • `print("#" * jumlah)` untuk grafik: soal meminta `for` dan `range()`. • Menyimpan semua kendaraan ke dalam list lalu mencari maksimum: list dilarang dan tidak perlu, karena nilai ekstrem bisa diperbarui langsung tiap kendaraan. • Menulis rumus tarif dua kali di `elif`: diganti pola hitung-lalu-potong agar sama dengan tabel.